MIC2026 Review Committee

Dear Members of the MIC2026 Review Committee,

I am writing to express my sincerest gratitude for the invaluable feedback provided by the reviewers on my paper submitted to MIC2026. Their insightful recommendations have significantly enhanced the quality and clarity of my work.

I have carefully incorporated all of the suggestions and feedback into my paper. In the following pages, you will find a detailed response to each comment and recommendation made by the reviewers. These revisions have strengthened my research's overall impact and contribution.

Once again, I would like to thank the reviewers for their time, effort, and expertise in evaluating my paper. The improvements made will enhance the quality of my contribution to MIC2026.

Thank you for considering my work for the presentation at MIC2026. I look forward to presenting my research and engaging with the conference attendees.

Sincerely,

DANIEL GIRALDO HERRERA

Corresponding Autor Paper 82

In the paper, the authors propose a Biased Random-Key Genetic Algorithm with Rolling Horizon (BRKGA-RH) to solve the Meal Delivery Routing Problem (MDRP), a dynamic and stochastic variant of the Vehicle Routing Problem arising in food delivery platforms. The authors introduce two variants of the algorithm: the former focused on maximizing the number of served orders, and the latter focused on minimizing travel time. These methods are compared against two baselines, a heuristic, and evaluated on 24 real-world instances from a delivery platform.

The paper discussed is interesting, but there are some concerns about the manuscript that need to be addressed:

Comment 1. *A relevant modeling doubt concerns the treatment of order coverage exclusively as an objective function rather than as a constraint. In practical meal delivery systems, service-level requirements often impose that all (or a very high percentage of) orders must be fulfilled within the planning horizon. Please discuss this choice in the manuscript.'*

> **Response:** Imposing such a constraint directly on the rolling-horizon problem creates two fundamental difficulties in a real-time setting. First, whether a target coverage can be achieved depends on future orders and courier availability that are unknown at dispatch time; a constraint that is feasible at the end of the day may be locally infeasible at every intermediate window. Second, the real instances used in this study contain large, high-volume orders in which even the best deterministic and evolutionary methods achieve 34–46% — values that reflect structural limitations of the available courier fleet rather than algorithmic shortcomings. Imposing coverage as a constraint above this ceiling would again render the problem infeasible. Treating coverage as an objective avoids these pitfalls: it always yields a feasible dispatch, and the algorithms maximize it subject to the hard operational constraints (capacity, time windows, eligibility) of Section 3.2. This choice is consistent with the benchmark literature (Reyes et al. 2018; Ulmer et al. 2021), where coverage or delivery rate is reported as the primary outcome metric, not a constraint.
>
> **Change in manuscript:** A paragraph discussing this modeling choice has been added at the end of Section 3.3.

Comment 2. *Is the coverage rate considered as an objective function computed for each time window or for the entire instance?*

> **Response**: Inside the BRKGA decoder, ρ is computed for the assignments produced in the current window w $\rho_w = |A_w|/|O_w|$ and is used as the primary component of the fitness functions for BRKGA-coverage and BRKGA-travel. This per-window value guides the evolutionary search within each 2-minute budget. The ρ values reported in Table 2 and discussed throughout Section 5 are computed over the entire planning day: $\rho = |\bar{O}|/|O|$, where $|\bar{O}|$ is the set of all orders successfully delivered by the end of the 1,440-minute horizon. This is the primary performance metric for comparing methods.
>
> **Change in manuscript:** The end of Section 3.3 and the opening of Section 5 have been clarified to distinguish the per-window fitness use from the end-of-day reporting metric.

Comment 3. Reviewer: Please explain better the structure of the chromosome.

Response: The chromosome is a real-valued vector of length nOw + nCw partitioned into two segments: (i) order keys (x1, . . . , xnOw): one real value in [0, 1] per pending order; the decoder sorts orders by these keys in descending order, so an order with a higher key is attempted for assignment before orders with lower keys. (ii) courier keys (xnOw +1, . . . , xnOw +nCw): one real value per available courier; the decoder tries couriers for each batch in descending key order, so higher-key couriers are preferred. The combined ranking induced by the two segments fully determines the greedy assignment produced by the decoder (Algorithm 2), without requiring any repair operator to restore feasibility.

Change in manuscript: We have clarified the structure through a numerical example, which has been added on page 9.

Comment 4. *One of my main doubts concerns the definition of the parameters (individual, probability, population size, delta, etc). How are they selected?*

Response: There are two different kind of parameters the platform stablishes three of them, the time window ($\Delta = 2\ min$) matches the platform dispatch cycle; the vehicle capacity limit is three orders per route.

The BRKGA hyperparameters ($N_p, G_{\max}, N_e, N_m, p_e$) were defined through a two-step process: first, standard values from the literature proposed by José Fernando Gonçalves and Mauricio Resende (2011) were used as a reference, including an elite inheritance probability of 0.7 and typical proportions for elite and mutant individuals relative to the population size; then, these values were jointly fine-tuned to ensure that the worst-case runtime—on very large instances—did not exceed 120 seconds per window on the available hardware. Since the main computational cost comes from the decoder, whose complexity grows proportionally with $N_p$and $G_{\max}$, it was observed that reducing these parameters below 10 led to poorer solution quality, while increasing them resulted in excessive computation times. Therefore, the final configuration $N_p = 10$, $G_{\max} = 10$, $N_e = 2$, $N_m = 2$, and $p_e = 0.7$ was selected, representing an optimal trade-off between solution quality and runtime reliability.

Change in manuscript: We describe better the parameters in section 4.1 after Algorithm 1 (page 9).

Comment 5. *Why does the time remaining after the end of the genetic loop remain unused? Why don't you also use a maximum number of generations as a stop criterion?*

Response: The $G_{\max}$= 10 is the primary stopping criterion: the genetic loop runs for exactly 10 generations and then stops, regardless of how much wall-clock time has elapsed. The 2-minute wall-clock budget Δ is a safety ceiling, not the loop termination condition; Algorithm 3 in the

manuscript notes that the best solution available at deadline Δ would be dispatched if the loop were to overrun — but our parameterization ensures this never happens in practice. However, the parameterization is designed so that this situation does not occur in practice. If the algorithm completes the $G_{\max}$generations before the 2-minute limit, the solution is immediately returned, allowing the system to respond to the user much earlier than the maximum allowed time and preventing unnecessary increases in waiting times for order assignment.

Change in manuscript: Algorithm 3 and its surrounding text have been revised to clearly state that Gmax is the stopping criterion and that the wall-clock budget serves only as a hard safety ceiling. A sentence on why remaining time is not reused has been added.

Comment 6. *Were the instances used an openly available dataset?*

Response: The dataset and the models implemented in this article are in the next github repository: https://github.com/Danielgiraldosb/BRKGA-for-Meal-delivery-routing-problem

Change in manuscript: The phrase "The implementation of the proposed approach is available in the following GitHub repository: https://github.com/Danielgiraldosb/BRKGA-for-Meal-delivery-routing-problem" was added to the article before section 5 Computational Experiments and Results

Comment 7. *In Table 2, the execution times in several cases are far superior to the window size of 2 seconds. Please clarify this point.*

Response: The execution times reported in Table 2 are total daily execution times (in seconds), accumulated over all rolling-horizon windows of a full 24-hour operating day — not per-window times. A planning day of 1,440 minutes with Δ = 2 minutes contains up to 720 windows; summing the per-window runtimes (each ≤ 120 s) naturally yields totals in the range of hundreds to thousands of seconds for large instances. For example, Instance 0 (56,420 orders) has TBC = 3,143 s, which corresponds to an average of 3,143/720 ≈ 4.4 seconds per window — well within the 120-second per-window budget. The 2-minute wall-clock constraint is therefore respected on every window; Table 2 simply aggregates these times over the day for efficiency comparison.

Change in manuscript: The caption of Table 2 and the paragraph introducing it in Section 5.1 have been updated to explicitly state that T values are total daily execution times and to note the per-window average for clarity.

Comment 8. In the abstract, you state that our approach outperforms the Business Model Lookahead and the Business Model by eight and fifteen points; while at the end of the introduction, you state that the percentage is "eleven to thirteen". Please check these sentences.

Response: The figures are not contradictory, they refer to different reference methods and instance scopes but the introduction text contained an error in the method name.

Change in manuscript: We avoid the outperformances from the introduction text to give more emphasis in the concepts and to make more space for others sections that needed revision as you suggested.

Comment 9. Please clarify the definition of ghost kitchens used in the literature review.

Change in manuscript: A brief definition of ghost kitchens has been added in Section 2.

Comment 10. Missing space in “availability [3].Static”

Response: The missing space in “availability [3].Static” has been corrected in the revised manuscript.

Comment 11. Missing dot in “arrivals [17, 13] Although”

Response: The missing period in “arrivals [17, 13] Although” has been corrected in the revised manuscript.

Comment 12. Please also add the specs of the CPU.

Response: We add the specs of the CPU and the language program we implemented in Section 5 second paragraph.

Comment 13. How were the algorithms implemented? (i.e. python, c++, java, etc)

Response: We add the specs of the CPU and the language program we implemented in Section 5 second paragraph, that correspond to the actual architecture of the server of the Latin American company.